

Fulton Hogan Assignment

Software Modelling FINAL Report ENSE706

Fateh Bhular 23217987



Summary from Progress Report

The first phase of this assignment focused on the initial planning of the new Project Costing & Management system of Fulton Hogan. We focused on the three most important sub-departments which were **Project Management, Finance, and Operations**. Each of these departments have three important stakeholders each:



Project Management	Finance	Operations
Project Manager Project Coordinator Site Lead	Financial Controller Auditor Project Accountant	Site Foreman Heavy-Machine Operator General Labourer

During requirements elicitation, two stakeholders from each sub-department were interviewed about their current system “JD Edwards”. It was noted that each user faced issues that reduced productivity or halted work all together. After all the interviews were finished, it was determined that their existing system suffered from data latency, a lack of real-time synchronisation, and data inconsistencies.

With the interviews, and some observations of JD Edwards online, the results were translated into functional and non-function requirements. These requirements were analysed thoroughly and focused on the main issues of the existing system being data integrity, system consistency, and communication between/in departments.

During UML Requirements Modelling, the system’s behaviour was implemented using a **Use-Case Diagram**. This diagram mapped the relationships between actors-actions, and actions-actions. The final iteration of the user-case diagram was used as a blueprint for the **Class Diagram** later on, which focused on the structural frame of the system program. This diagram converted the different use-cases into concrete and abstract classes with labeled interactions. The last iteration of the class diagram was a great improvement over the first iteration, as more classes were added to support previous inconsistencies. Design principles such as SRP, ISP, etc were improved throughout the iterations, and abstract classes were also improved.

During the iterations of the use-case and class diagrams, the **C# prototype** was being produced. The initial iteration was focused on the main concrete classes that were stated in the use-case descriptions section. During production, design risks were identified, such as high coupling, low cohesion, not following SRP, and more. The Agile cycle was implemented because it was the right fit, allowing improvements to be made during development and not at the end when everything is almost finished.

This class diagram also showcased **high coupling** and **low cohesion** between classes. This means that some classes were very dependent on other classes, while some classes had more than one responsibility. These are very severe system problems that become harder to handle the more you scale and maintain the system. With high coupling, modifying a class will affect

SOLID principles have been focused on, however there are some ends of the diagram that need polishing. *GeneralLabourer* and *HeavyMachineOperator* have the system login and logout methods that are inherited from the *User* class. This violates the **Liskov Substitution principle** because those two classes inherit from *User* but have different behaviours. Those two sub-classes are employees who never use the main system, so substituting them in-place of *User* could cause a system error. The **Open/Closed principle** is being violated by the *User* class. Because *User* is a concrete class, it has implementations, and the methods are defined. If we wanted to add a new type of employee to the system, *User* would have to be modified, thus violating OCP.

[illegible]

Figure 2: Refined UML Class Diagram

SOLID. The *ReportService* and *ReportFactory* classes (Figure 2) are the result of splitting the legacy *Report* concrete class. Now every employee accesses *ReportService* which is responsible for processing reports, and that can access *ReportFactory* which creates reports. This solves the SRP violation in the legacy class diagram, where data processing and creation was cluttered into a single concrete *Report* class. The new *ReportService* and *ReportFactory* also follows the **high cohesion** principle that is very important in software system design. The relationships between different roles and objects have been decoupled using mediator objects, e.g. *TaskListManager* and *ReportService*. *ProjectCoordinator* and *SiteLead* now interact with *TaskListManager* instead of directly to *TaskList*, which decreases the coupling between the two classes following **low coupling** principle. This allows for modifications to be made in an isolated environment, so that other classes are not affected by the new changes.

1.2 Sequence Diagrams

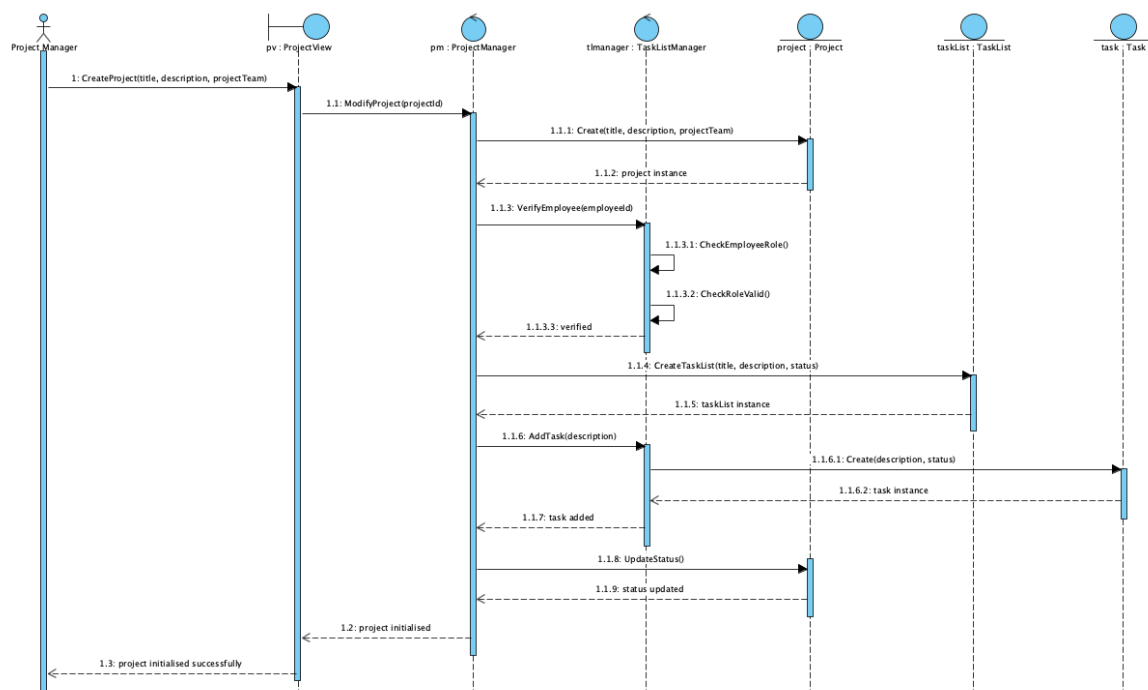


Figure 4: Sequence Diagram for “Modifying a project”

The sequence diagram in Figure 4 models a *ProjectManager* employee **modifying a project**. This diagram uses the Boundary-Control-Entity pattern, where all three lifelines have a responsibility in this sequence of actions. The *ProjectManager* actor interacts with the *ProjectView* boundary, which separates the complex, behind-the-scenes logic from the user. There is a *ProjectManager* control lifeline which is the primary controller. Responsibility of project modification is split between *ProjectManager* which creates the project, and *TaskListManager*, which handles task-related jobs.

Before any important modifications are made, the *ProjectManager* control lifeline runs self-messages to check if the user has the correct privileges. This is very important for security reasons so that someone at a lower level cannot change something crucial about the project.

This sequence diagram handles the instantiation of three objects being *Project*, *TaskList*, and *Task*. The *ProjectManager* control entity creates a project, then gives control to *TaskListManager* which creates the task list and tasks. Once the instantiation is complete, the messages flow back up through the lifelines, and at the end, a message “project initialised successfully” confirms that each lifeline completed its responsibility. The final result is reported to the *ProjectManager* actor via the *ProjectView* boundary.

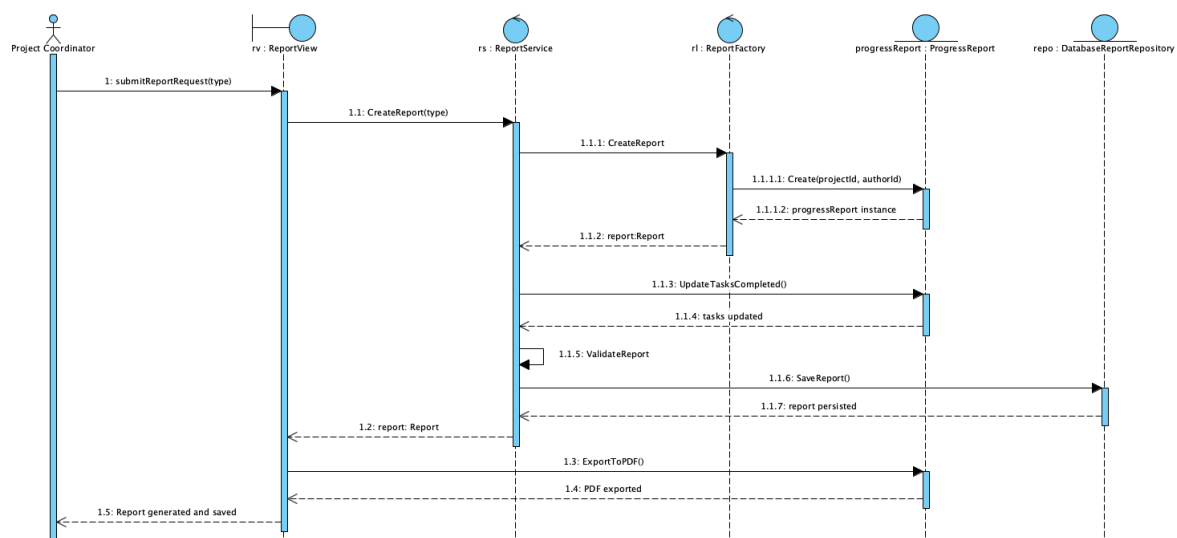


Figure 5: Sequence Diagram for “Generating a Progress Report”

The sequence diagram in *Figure 5* models a *ProjectCoordinator* **generating a progress report**. This diagram also uses the Boundary-Control-Entity pattern, where *ReportView* acts as a boundary separating the *ProjectCoordinator* from the complex system logic. *ReportService* is the primary controller and *ProgressReport* and *DatabaseReportRepository* are the entities.

A design choice in this diagram was to add *ReportFactory*. Instead of *ReportService* being responsible for both data handling and creation, the responsibility is now split up into two classes which follow the **Single Responsibility Principle**. The factory handles the creation call and returns an instance back to the service, which keeps object creation logic separate from business logic.

This sequence diagram handles the instantiation of a *ProgressReport* entity. Because *ReportService* is responsible for business logic, the creation is handled by *ReportFactory*,

which returns a *ProgressReport* instance. After creation, *ReportService* uses methods to modify the progress report and save it to *DatabaseReportRepository*. After the report has been exported as a PDF, the messages go back up through the lifelines, and a message “Report generated and saved” confirms the completion of this sequence.

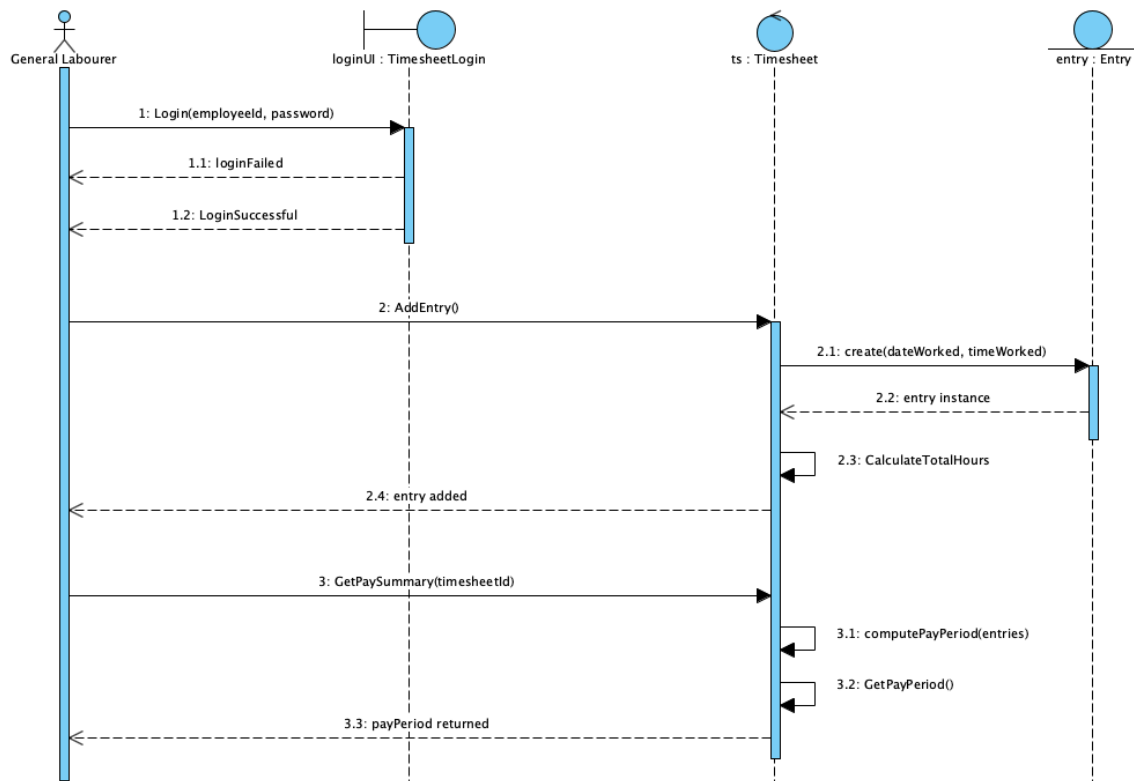


Figure 6: Sequence Diagram for “Creating a timesheet entry”

The sequence diagram in *Figure 6* models a *GeneralLabourer* **creating a timesheet entry** on the timesheet system. This diagram uses a Boundary-Control-Entity pattern, where all three lifelines have a responsibility. The *GeneralLabourer* actor interacts with the *TimesheetLogin* boundary which separates the system logic from them. The *Timesheet* control entity is the primary controller that handles timesheet operations. Responsibility of creating an entry is split between *Timesheet* and *Entry* where *Timesheet* manages overall timesheet logic, and *Entry* handles the creation of entries.

Before any entries are made, the *TimesheetLogin* boundary validates the *GeneralLabourer*’s credentials before allowing them into the timesheet system. If the login succeeds, then the *GeneralLabourer* will be able to add an entry and get pay summaries. If the login fails, they will be told that their credentials are incorrect. This was implemented for the purpose of security, as we don’t want an outsider logging into an employee’s timesheet.

This sequence diagram handles the instantiation of the *Entry* entity using the *create()* method from *Timesheet*. Once the entry is added, the *GeneralLabourer* can request a pay summary

which triggers two self-messages (1) computing the payperiod using current entries (2) gets the pay period. Instead of making the *Entry* entity handle logic, *Timesheet* handles the logic since it is the controller. The return messages flow back through the lifelines, with the message “payPeriod returned” confirming that the sequence is complete.

1.3 Activity Diagrams

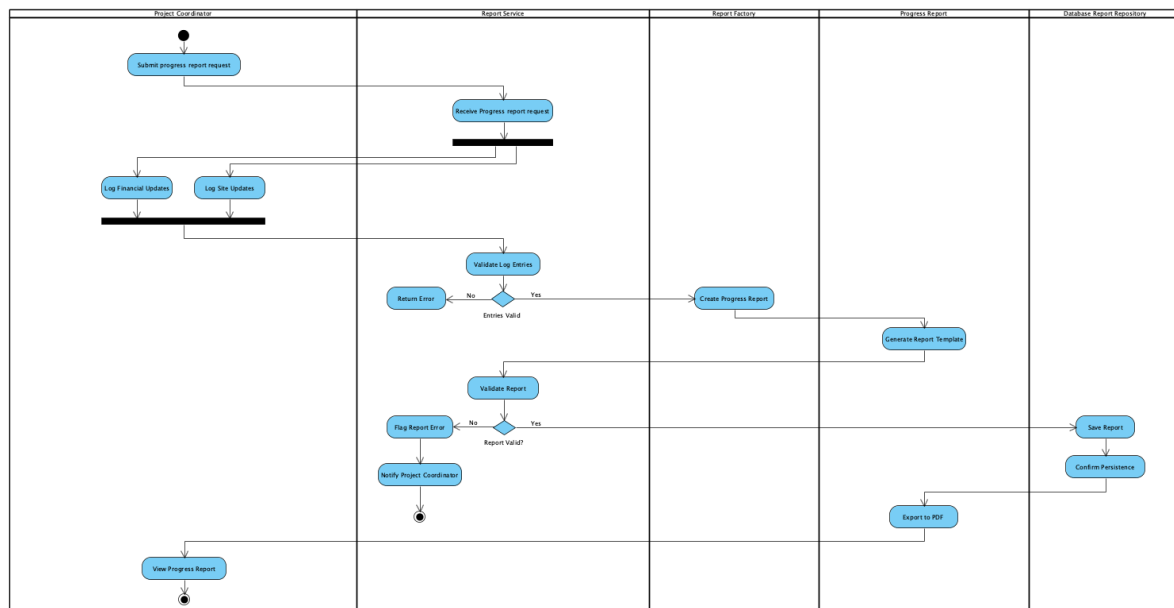


Figure 8: Activity Diagram for “Logging Progress and Generating Report”

The activity diagram in *Figure 8* models the flow of progress report generation. There are 5 swimlanes each representing a class or object that has a responsibility in this flow of events. A fork node and join node were used to show the options a user has. They can choose to log financial or site updates into the report. This is joined later and validated by the ReportService class. For conditions, the decision node was used to show the options that could occur as a reaction to an event (like an if-else statement). For example, validation of the report had 2 different outcomes (1) If not validated, then the user would be flagged (2) If validated, the report would be saved into the database. If the report was persisted into the database, a PDF would also be generated for easy viewing for any viewers. Both fork, and decision nodes are important to a sequence diagram, because it allows for diagrams that are easier to understand. Outsiders wouldn’t know what an if-else statement is, so these nodes display it in a way that is easy to visualise.

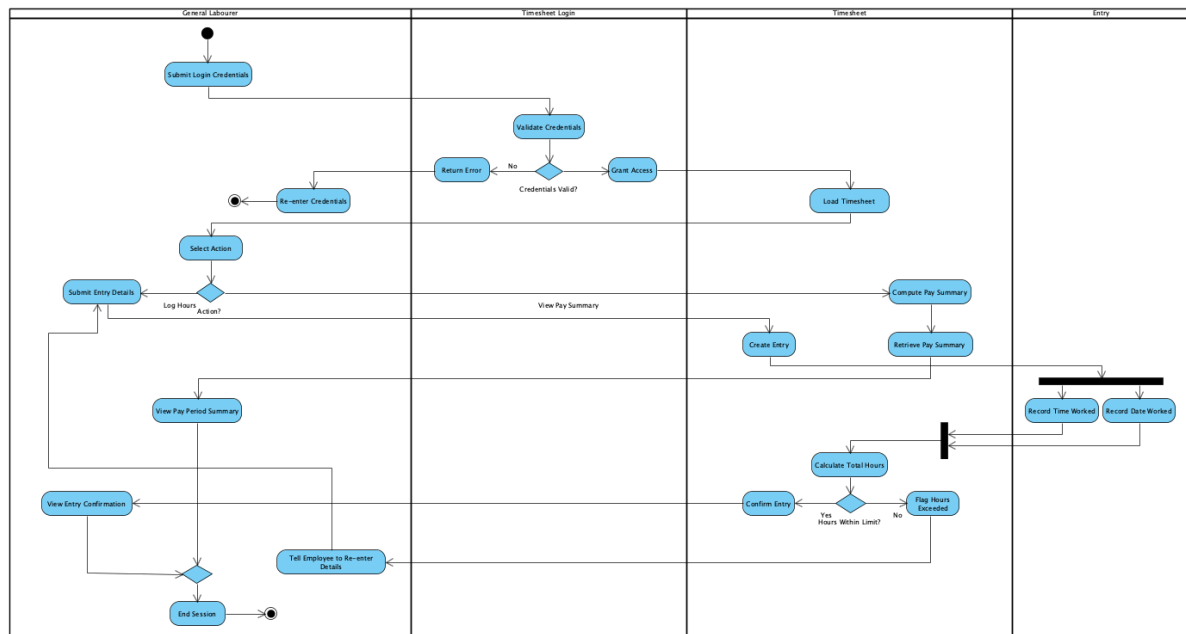


Figure 9: Activity Diagram for “Timesheet Operation Flow”

The diagram in *Figure 9* models the Timesheet Operation Flow, which covers both the Log Hours Worked, and View Pay Period Summary use-cases (*Figure 4*). There are four swimlanes representing General Labourer, TimesheetLogin, Timesheet, and Entry. All four of these objects/classes play a role in this operation flow. A decision node was used for validating credentials because there are two possible cases (1) employee enters correct details, so they can access timesheet (2) employee enters incorrect details, they are prompted to sign in again. Because there are two different outcomes from the Validate Credentials action, a decision node is more suitable than a fork node. However, a fork node was used to create an Entry. Because the system has to record two things that are separate, it could occur at the same time to save time. When both actions are completed, they go into a join node and the system can then perform calculations.

2. Design Pattern Application and Implementation

Github Link:

<https://github.com/fatehbhular/FultonHogan-FatehBhular-23217987>